

CAP 931: Capstone - Build a Sales Agent Prototype Using Multi-Agent Models

1. Project Title & Overview

- **Project Name:** Multi-Agent Sales Assistant Prototype ([sales-agent-prototype](#))
 - **Course/Capstone:** CAP 931 Capstone Project
 - **Author:** Franky Achu Ndalle Jr.
 - **Objective:** Build an intelligent multi-agent sales assistant prototype powered by Large Language Models and Streamlit to help enterprise sales representatives analyze target accounts, evaluate competitor overlaps, perform live source-grounded web scraping via BeautifulSoup, and generate executive-ready account insight one-pagers.
-

2. Technical Foundation & Environment Setup (Step 1)

To establish a robust development environment for rapid prototyping, the following technical stack and local configurations were implemented:

Programming Language: Python 3.10+

Frameworks & Libraries:

1. [streamlit](#) (For front-end interactive UI development)
2. [langchain-groq](#) & [langchain-core](#) (For orchestrating multi-agent chains and prompt templates)
3. [python-dotenv](#) (For secure environment variable handling)
4. [beautifulsoup4](#) & [requests](#) (For live web-scraping and source-grounded data ingestion)
5. [uv](#) (Modern Python package manager and dependency resolution)

Environment Initialization Commands (uv Workflow):

Bash

- `uv sync`
- `uv run streamlit run main.py`

Security Configuration: API keys are securely isolated via a local `.env` file (`GROQ_API_KEY`) and excluded from Git version control using `.gitignore`.

UI Initiation: Tested and launched locally via `uv run streamlit run main.py`, successfully compiling the server at <http://localhost:8502/>.

3. Dynamic Inputs & Data Collection Layer (Step 2)

The user interface features a reactive Streamlit sidebar that accepts configurable account metadata, allowing sales representatives to dynamically input campaign parameters:

- **Product Name:** The software/solution being pitched (e.g., Enterprise Cloud Data Platform).
 - **Value Proposition:** Core advantage statement (e.g., Reduces data warehousing costs by 40% with automated scaling).
 - **Target Company URL:** The prospective account domain (e.g., [\[https://example-enterprise.com\]](https://example-enterprise.com)(<https://example-enterprise.com>)).
 - **Product Category:** Industry or technical classification (e.g., Data Warehousing).
 - **Competitor Names / URLs:** Market rivals for comparative evaluation (e.g., Snowflake, Databricks).
 - **Target Customer / Persona:** Decision-maker role (e.g., Chief Data Officer).
-

4. LLM Capabilities, Model Selection & Data Integration (Step 3)

- **Model Engine Selected:** Groq API running `openai/gpt-oss-120b` via LangChain's `ChatGroq` wrapper.
 - **Technical Justification:**
 1. **Speed:** Groq's high-performance inference engine provides near-instantaneous token generation, enabling seamless multi-agent prompt chaining without latency.
 2. **Reasoning Quality:** Offers advanced contextual reasoning, structured parsing, and strategic synthesis for enterprise sales workflows.
 - **Live Web Data Integration & Source Grounding:**
 1. The application incorporates a custom web-request and HTML-parsing function (`fetch_web_content`) utilizing `BeautifulSoup` and `requests` to fetch live text directly from target company and competitor URLs.
 2. The Research Agent system prompt explicitly instructs the model to ground its analysis strictly in the retrieved web evidence and highlight unverified claims, eliminating LLM hallucinations.
 - **Prompt Chaining Architecture (`main.py`):**
 1. **Sales Intake Agent:** Parses target account requests to identify core customer goals, use cases, and technical requirements.
 2. **Market Research Agent:** Evaluates live scraped web evidence and direct competitor overlaps against specified rivals.
 3. **Strategy Synthesis Agent:** Consolidates prior agent outputs into a unified executive one-pager complete with source verification links.
-

5. System Outputs & Executive One-Pager Deliverable (Step 4 & 5)

When executed through the Streamlit interface, the application generates a structured one-pager aligned with Per Scholas evaluation requirements:

- **Executive Summary:** Summarizes organizational activities and customer goals.
- **Competitor Analysis:** Evaluates direct overlaps against specified rivals (e.g., Snowflake, Databricks).

- **Key Leadership Focus Areas:** Identifies key executive priorities across product development and customer success.
 - **Source Verification:** Includes direct references and target company URLs for fact-checking.
 - **Tailored Pitch Strategy:** Delivers actionable messaging emphasizing total cost of ownership reduction and automated scaling.
-

6. Optional Enhancements

- **Improving Output Results:** Utilizing multi-agent prompt chaining (`intake_output` → `research_output` → `synthesis_output`) to isolate agent responsibilities and enhance structural depth.
 - **Alert System:** Designed background cron-job/Celery architectures to target RSS feeds or job boards.
 - **Production Deployment:** Containerized via Docker and deployed to cloud environments (AWS ECS or Streamlit Cloud).
-

7. Documentation

A. Technical Documentation & Code Structure (`main.py`)

B. Time Management Allocation

- **Phase 1 (~20%):** Configuring Python environments, initializing `uv`, and setting up secure local environment variables.
- **Phase 2 (~50%):** Designing specialized agent roles, BeautifulSoup web scraping integration, and multi-agent prompt chaining.
- **Phase 3 (~30%):** Validating local execution via `uv run streamlit run main.py` and finalizing project documentation.

C. Challenges and Solutions

- **Challenge:** Ensuring LLM research outputs remain grounded in actual web data without hallucinating company background details.
- **Solution:** Implemented a live BeautifulSoup extraction pipeline that feeds raw text directly into the Research Agent prompt with strict system instructions.

D. Experiments & Outcomes

- **Experiment:** Tested standalone model prompts versus a multi-agent source-grounded architecture.
- **Outcome:** Splitting logic into dedicated autonomous agent roles with BeautifulSoup data ingestion dramatically improved factual accuracy, relevance, and professional formatting.